SAFE-STRIDE ASSIST – AN AI-BASED SAFE WALKING ASSISTANCE SYSTEM FOR PEOPLE WITH VISUAL IMPAIRMENTS

A PROJECT WORK REPORT SUBMITTED TO

THE NATIONAL INSTITUTE OF ENGINEERING

(An Autonomous Institution Under VTU)

In fulfillment of the requirements for major project work, Seventh semester
Bachelor of Engineering In Computer Science & Engineering
Submitted By

Jhenkar P – [USN]

Under The Guidance Of

[Guide name]

Assistant Professor

Department of CS&E,
NIE Mysore

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
THE NATIONAL INSTITUTE OF ENGINEERING
(An Autonomous Institution Under VTU)
No.50 (Part), Koorgalli village, Hootagalli Industrial Area,
Mysuru-570008 Karnataka
2026-2027

SAFE-STRIDE ASSIST – AN AI-BASED SAFE WALKING ASSISTANCE SYSTEM FOR PEOPLE WITH VISUAL IMPAIRMENTS

THE NATIONAL INSTITUTE OF ENGINEERING
(An Autonomous institution, affiliated to VTU)
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
Mysore -570008

CERTIFICATE

This is to certify that the project work entitled **"Safe-Stride Assist: An AI-Based Safe Walking Assistance System for People with Visual Impairments"** is a bonafide work carried out by **Jhenkar P** in fulfillment of the requirements for the major project work of the Bachelor of Engineering in Computer Science and Engineering, Seventh Semester, at The National Institute of Engineering (Autonomous under VTU), Mysuru, during the academic year **2026–2027**.

It is certified that all corrections and suggestions indicated during the Internal Assessment have been incorporated in the report deposited in the department library. The major project work report has been approved in fulfillment of the academic requirements prescribed by The National Institute of Engineering, Mysuru.

Signature of Internal Guide	Signature of HOD	Signature of Principal
[Guide name]	Dr. Anitha R	[Principal name]
Assistant Professor	Professor and Head	Principal,
Dept. of CS&E	Dept. of CS&E	NIE, Mysuru
NIE, Mysuru	NIE, Mysuru	

Name of Examiners **Signature and Date**

ABSTRACT

People with visual impairment depend on a white cane, a guide dog and help from other people to move safely, but these aids give little information about distant, overhead or unfamiliar obstacles. Most camera-based assistants stop at drawing bounding boxes around objects: they do not say whether an object blocks the walking path, how far away it is, or what the user should do. To address this gap, this project presents **Safe-Stride Assist – An AI-Based Safe Walking Assistance System for People with Visual Impairments**, a research and teaching prototype that turns a single camera stream into short, prioritised spoken guidance.

The system processes camera frames locally. YOLO11s detects walking-relevant objects, a second YOLO11s model detects street hazards such as stairs, poles, pillars and cones, and ByteTrack keeps object identities across frames. Depth Anything V2 estimates a depth map from the single image. A spatial reasoning layer back-projects the depth map into three-dimensional points, fits the ground plane with RANSAC, uses the known camera height to convert depth to approximate metres, and tests a one-metre-wide, three-metre-long walking corridor, so that unknown blockers are found even when no object class exists for them. Looming-based time-to-collision, persistence checks and cooldowns decide what is spoken and when, for example “Stop! Path blocked, 3 meters. Keep Left.”

A route-guidance module combines GPS, a compass-derived camera heading and OpenStreetMap foot routing to give turn-by-turn instructions with off-route detection and arrival notification. The system is delivered as a Python back-end with a browser console and an Android client built with Expo and React Native.

The system was verified with 41 automated tests and with controlled and real-video experiments. In controlled scenes the ground-plane scale correction kept the mean distance error at 2.4 cm even when the raw depth was twice too large, while the uncorrected pipeline still detected only 25 % of the blockers. The six-class hazard detector reached 0.556 mAP50 on 540 held-out images, and the full pipeline ran at about 20 frames per second on a laptop GPU. The prototype is not a certified mobility device and must be used together with a cane, a guide dog or human support.

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of people whose ceaseless cooperation made it possible whose constant guidance and encouragement crown all efforts.

First and foremost we would like to thank our beloved principal **[Principal name]** for being the patron and the beacon light for this project.

We would like to express our sincere gratitude to our HOD **Dr. Anitha R**, Department of CSE, NIE for her relentless support and encouragement. It gives us immense pleasure to thank our guide **[Guide name]**, Assistant Professor, Department of CSE, NIE for the valuable suggestions and guidance during the process of the project and for having permitted us to pursue work on the subject.

Jhenkar P ([USN])

CONTENTS PAGE NO

1. Introduction 1
1.1. Overview 1
1.2. Objective 1
1.3. Scope of the Project 2
1.4. Existing System 2
1.5. Proposed System 3
2. Literature Survey 5
3. System Requirements and Specifications 8
3.1. Software Requirements 8
3.2. Hardware Requirements 8
3.3. Functional Requirements 9
3.4. Non-Functional Requirements 9
4. System Design and Analysis 11
4.1. High Level Design 11
4.2. Low Level Design 12
4.3. Architecture Overview 16
5. Implementation 18
5.1. Implementation 18
5.2. Key Parameters 23
5.3. User Interfaces 24
6. Testing 25
6.1. Types of Testing 25
6.2. Test Cases 26
6.3. Defects Found and Fixed During Testing 28
7. Results and Discussions 30
7.1. Hazard Detector Results 30
7.2. Effect of Ground-Plane Scale Correction 31
7.3. Time-to-Collision Accuracy 32
7.4. Processing Speed 33

7.5.	Behaviour on Real Walking Video	33
7.6.	Route Guidance under GPS Noise	34
7.7.	Overall System Output	35
7.8.	Discussion	36
7.9.	Conclusion	37
7.10.	Future Scope	37
8.	References	38

Figure No	Figure Name	Page No
Fig 4.1	High level design	11
Fig 4.2	System components	12
Fig 4.3	Use case diagram: walking assistance	13
Fig 4.4	Use case diagram: route guidance	13
Fig 4.5	Data flow diagram	14
Fig 4.6	Sequence diagram	15
Fig 4.7	Walking corridor geometry	15
Fig 4.8	Navigation state diagram	16
Fig 7.1	Hazard detector training curves	30
Fig 7.2	Normalised confusion matrix	31
Fig 7.3	Per-class average precision	31
Fig 7.4	Effect of scale correction	32
Fig 7.5	Annotated frame and depth map	34
Fig 7.6	Failure case: false stairs detections	34
Fig 7.7	Browser console	36

Chapter 1

INTRODUCTION

1.1 Overview

Moving safely through streets, footpaths and buildings depends on knowing what is ahead. People with visual impairment combine a white cane, a guide dog, memory of the environment and help from other people to do this, and these methods remain essential. A camera, however, can add another source of information about things that a cane reaches late or not at all, such as head-level obstructions, approaching vehicles and steps.

Safe-Stride Assist is an AI-based safe walking assistance system that analyses a camera stream locally and produces short spoken guidance. It combines YOLO11s object detection, ByteTrack tracking and Depth Anything V2 monocular depth estimation with a geometric reasoning layer that estimates the ground plane, converts depth to approximate metres and evaluates a one-metre-wide, three-metre-long walking corridor. The result is a message such as "Stop! Path blocked, 3 meters. Keep Left." instead of an unlabelled bounding box.

The project also includes a route-guidance module that uses GPS, a compass-derived camera heading and OpenStreetMap routing, a browser console for monitoring, and an Android client. It is a research and teaching prototype; it is not a certified mobility, medical or emergency device.

1.2 Objective

The objective of Safe-Stride Assist is to develop a supplementary walking-assistance prototype that gives path-aware, prioritised guidance from a single camera. The specific objectives are:

- To build a real-time computer-vision pipeline for walking assistance using a webcam, an uploaded video or a phone camera.
- To detect walking-relevant objects with YOLO and to track them over time with ByteTrack.
- To estimate approximate object distance from monocular depth and ground-plane calibration.
- To analyse a three-metre forward walking corridor and decide whether it is blocked.
- To detect unknown blockers from depth points that rise above the ground, without a trained class.

- To detect stairs and report whether they go up or down.
- To generate concise voice alerts containing the obstacle, distance, direction, urgency and a suggested side.
- To reduce false and repeated alerts using persistence checks, cooldowns and priority ranking.
- To provide GPS route guidance with heading alignment, off-route detection and arrival notification.
- To verify the system with automated tests and experiments and to document its limits honestly.

1.3 Scope of the Project

The scope of the project covers the complete path from a camera frame to a spoken message. It includes object detection and tracking, monocular depth estimation, ground-plane scale correction, corridor analysis, stair detection, time-to-collision estimation, alert ranking, speech and beep output, a browser console, an Android client and route guidance over OpenStreetMap data. Messages can be generated in English and French (French is selected in the video-processing script).

The following are outside the scope of the current work: certification as a mobility or medical device, a study with visually impaired users, haptic hardware, indoor map navigation, and training on a large labelled walking dataset. These are listed as future work in Chapter 7.

1.4 Existing System

Existing camera-based assistance demonstrations commonly use object detection to draw labels and bounding boxes over a live image. Some add monocular depth or tracking, but these components are often presented independently instead of being combined into one walking-path decision. Generic detectors cannot identify every real-world hazard, and a detector label alone does not indicate whether an obstacle blocks the user's path or how urgently it should be announced. Route-guidance applications, in turn, guide the user between places but say nothing about a branch, a step or a temporary barrier on the footpath.

1.4.1 Drawbacks

- **Limited reach of manual aids:** a white cane or guide dog remains essential, but gives limited information about distant, overhead or unfamiliar obstacles.

- **Limited distance awareness:** a bounding box or label does not tell the user how urgently to respond.
- **No walking-corridor reasoning:** objects are reported whether or not they lie in the user's forward path.
- **Unknown blockers may be missed:** temporary structures and objects without trained classes may remain undetected.
- **Weak temporal reasoning:** frame-by-frame analysis cannot tell whether an obstacle is approaching or crossing.
- **Repeated and low-value alerts:** frequent announcements raise cognitive load and hide urgent information.
- **Separate obstacle and route systems:** obstacle sensing and route guidance are usually different products.
- **Unclear safety boundaries:** monocular depth is not a guaranteed distance, and experimental systems must say so.

1.5 Proposed System

The proposed Safe-Stride Assist system is a local, AI-based walking assistant that processes camera frames and generates concise voice guidance. It combines YOLO obstacle detection, ByteTrack temporal tracking, Depth Anything V2 monocular depth, RANSAC ground-plane fitting, a three-dimensional walking-corridor model, approach-risk estimation and prioritised speech. Known obstacles come from the detectors; unknown blockers come from depth points above the ground plane inside the corridor. A route module adds GPS guidance, and the same engine serves a browser console and an Android client.

1.5.1 Advantages

- **Distance-aware assistance:** relative monocular depth is converted into approximate metres using the camera height.
- **Known and unknown blocker reasoning:** named objects and depth-based blockers share one corridor decision.
- **Walking-path relevance:** objects outside the corridor do not trigger path warnings.
- **Actionable feedback:** a clearer side (left or right) is suggested when the geometry supports it.
- **Reduced alert overload:** persistence, urgency, cooldowns and ranking keep speech

short.

- **Stairs awareness:** a hazard model finds stairs and the depth slope confirms up or down.
- **Route and obstacle guidance together:** GPS instructions and local warnings come from one system.
- **Local and privacy-oriented:** camera frames are processed on the user's own computer, not in a cloud service.
- **Modular and testable:** detection, depth, geometry, risk, speech and navigation are separate modules with automated tests.

Chapter 2

LITERATURE SURVEY

1. "You Only Look Once: Unified, Real-Time Object Detection" (2016) [1]

This work introduced YOLO, a one-stage detector that predicts boxes and classes for the whole image in a single network pass, which made real-time detection practical. However, a detector only reports what is visible and where it is in the image; it does not estimate distance, decide whether an object lies in the walking path, or choose what to say. Safe-Stride Assist uses a current Ultralytics YOLO11s model [5] as its perception front end and adds depth and path reasoning on top.

2. "ByteTrack: Multi-Object Tracking by Associating Every Detection Box" (2022) [2]

ByteTrack improves multi-object tracking by associating almost every detection box, including low-confidence ones, with existing tracks so that identities survive partial occlusion. However, tracking by itself gives identity, not distance or danger. Safe-Stride Assist uses ByteTrack identities to measure how an object's image size changes over time, from which it estimates time-to-collision, and to ignore one-frame false detections.

3. "Depth Anything V2" (2024) [3]

Depth Anything V2 is a monocular depth estimation model trained on synthetic images and on pseudo-labelled real images, and it is available with fine-tuned metric variants. It gives a dense depth map from a single camera frame. However, depth from one image is scene-dependent and can be wrong by a roughly constant factor, so it cannot be trusted as a distance without calibration. Safe-Stride Assist uses the Metric-Outdoor-Small variant and corrects its scale with the estimated ground plane and the camera height.

4. "Vision Transformers for Dense Prediction" (2021) [4]

This paper proposed dense prediction transformers, which use a vision-transformer backbone to produce dense outputs such as depth with a more global view of the image than convolutional networks. It showed the value of transformer-based monocular depth for scene understanding. However, a depth map describes the structure of a scene and does not by itself give a guaranteed real-world distance. This motivates the explicit scale correction and the safety limits in Safe-Stride Assist.

5. "Analysis of Navigation Assistants for Blind and Visually Impaired People: A Systematic Review" (2021) [7]

This systematic review analysed navigation assistants for blind and visually impaired people

and compared their technologies and limitations. It concludes that electronic aids are meant to complement, not replace, traditional mobility aids. However, a review does not provide a working system. Safe-Stride Assist follows its guidance by presenting itself as a supplementary aid and by combining perception, route guidance and safety feedback.

6. "Wearable Obstacle Avoidance Electronic Travel Aids for Blind and Visually Impaired Individuals: A Systematic Review" (2023) [8]

This review covers wearable obstacle-avoidance travel aids and discusses their sensing range and feedback methods. It points out that a visual label alone does not tell the user how urgently to react. However, many of the aids it covers depend on dedicated wearable hardware. Safe-Stride Assist aims to deliver distance-aware warnings with a phone camera or a webcam and a laptop GPU.

7. "Safe Local Navigation for Visually Impaired Users With a Time-of-Flight and Haptic Feedback Device" (2018) [9]

The authors built a wearable device that uses time-of-flight distance sensing and haptic feedback to support safe local navigation. It shows that distance-aware local warnings and feedback that does not depend on speech are valuable. However, it needs a purpose-built sensing and vibration device and does not name the obstacle. Safe-Stride Assist estimates distance from a camera and names the obstacle, and treats richer haptic feedback as future work.

8. "A Systematic Review of Urban Navigation Systems for Visually Impaired People" (2021) [10]

This review examines urban navigation systems for visually impaired people, including route guidance and location-aware assistance. It supports the use of destination search, turn instructions, heading alignment, off-route detection and arrival messages. However, route guidance alone does not describe local obstacles on the footpath. Safe-Stride Assist combines OpenStreetMap-based route guidance with camera-based obstacle warnings.

9. "The Sound of Vision Project: The Concept and First Results" (2016) [11]

The Sound of Vision project explored how information about the surrounding scene can be presented to blind users through sound and touch. It supports the use of spatial audio as a non-visual interface. However, it relies on a dedicated sensing and rendering device, and detailed audio scenes can overload the user. Safe-Stride Assist keeps its output deliberately short: a beep panned toward the obstacle followed by one sentence.

10. "Haptic Wearables as Sensory Replacement, Sensory Augmentation and

Trainer" (2015) [12]

This review studies haptic wearables as sensory replacement, sensory augmentation and training tools. It supports adding vibration cues when speech alone may be insufficient or distracting. However, it is a survey of devices and does not address camera-based perception. Safe-Stride Assist currently speaks and beeps; the web console also vibrates the device on urgent alerts when the browser allows it, and richer haptic cues are future work.

11. "Exploring Levels of Control for a Navigation Assistant for Blind Travelers" (2023) [13]

This study explores different levels of control that a navigation assistant can give to blind travellers and finds that preferences vary with the situation. However, it is concerned with interaction design rather than with the perception pipeline. Safe-Stride Assist keeps the user in control: it announces and advises but does not take over decisions.

12. "An ARCore Based User Centric Assistive Navigation System for Visually Impaired People" (2019) [14]

This work builds a user-centric assistive navigation system for visually impaired people on a smartphone using the ARCore augmented-reality framework. It shows that a phone can serve as a portable platform that combines camera, positioning and guidance. However, it depends on AR tracking support on the device. Safe-Stride Assist takes a different route: learned object detection and monocular depth with ground-plane scale correction, processed locally.

Research Gap

The surveyed work shows that detection, tracking, depth, route guidance and multimodal feedback have each been studied. What is less common is a single, locally running system that converts a detection into a decision about the user's forward path: where the object is relative to the walking corridor, how far away it is in metres, whether it is approaching, and which short message deserves to be spoken. Safe-Stride Assist addresses this gap by combining detection, tracking, scale-corrected monocular depth, corridor reasoning, risk ranking and route guidance, and by reporting its limits.

Chapter 3

SYSTEM REQUIREMENTS AND SPECIFICATIONS

3.1 Software Requirements

- **Operating System:** Windows 10/11 (server-side voice uses Windows SAPI); Android for the phone client.
- **Programming Languages:** Python 3.12 (3.10 or newer), JavaScript, TypeScript.
- **Deep Learning Framework:** PyTorch 2.5.1 with CUDA 12.4.
- **Detection and Tracking:** Ultralytics 8.4 YOLO11s and ByteTrack.
- **Depth Estimation:** Depth Anything V2 Metric-Outdoor-Small through Hugging Face Transformers 4.49.
- **Image Processing:** OpenCV 4.10 and NumPy 1.26.
- **Speech and Audio:** Windows SAPI (comtypes), sounddevice, imageio-ffmpeg; browser speech synthesis; expo-speech on the phone.
- **Back-end:** Python HTTP server (ThreadingHTTPServer) with JSON and MJPEG over HTTP.
- **Web Console:** HTML, CSS and JavaScript.
- **Mobile Client:** Expo SDK 58, React Native 0.88, expo-camera, expo-location, expo-sensors, expo-speech, react-native-webview with Leaflet.
- **Maps and Routing:** OpenStreetMap data, Nominatim and Photon place search, OSRM foot routing (Google Places optional).
- **Testing:** pytest for automated tests.
- **Model Training:** Ultralytics training on Kaggle (Tesla T4 GPU).
- **Development Tools:** Visual Studio Code, Git, Node.js and npm.

3.2 Hardware Requirements

- **Processor:** Intel Core i7 or an equivalent processor.
- **Memory (RAM):** 8 GB minimum; 16 GB recommended.
- **GPU:** NVIDIA GPU with CUDA support (the development machine uses a NVIDIA GeForce RTX 4060 Laptop GPU).
- **Storage:** 256 GB SSD or higher for project files, model weights and test videos.

- **Camera:** USB or IP webcam, or the camera of an Android phone.
- **Phone:** Android phone with GPS, accelerometer and magnetometer for route guidance.
- **Audio Output:** Speaker or headphones for voice alerts and directional beeps.
- **Network:** Local Wi-Fi between the phone and the computer; internet access for place search and routing.

3.3 Functional Requirements

- The system shall accept frames from a webcam, an uploaded video file or the phone camera.
- The system shall detect walking-relevant objects and hazards and track them across frames.
- The system shall estimate the approximate distance of each detected object in metres.
- The system shall estimate the ground plane and evaluate a 1 m × 3 m walking corridor.
- The system shall detect unknown blockers that rise above the ground inside the corridor.
- The system shall detect stairs and report whether they go up or down.
- The system shall estimate whether an object is approaching and its time-to-collision.
- The system shall rank alerts by urgency and speak only the most relevant one, avoiding repetition.
- The system shall give spoken and written messages with distance, direction and a suggested side.
- The system shall provide a browser console showing the annotated feed, path status and alerts.
- The system shall search a destination, create a walking route and give turn-by-turn guidance.
- The system shall warn when the user leaves the route and announce arrival once.

3.4 Non-Functional Requirements

- **Performance:** the pipeline should process frames fast enough for near real-time use on a laptop GPU.
- **Reliability:** the system should keep running through bad frames, missing depth, lost

GPS or a missing speech engine.

- **Maintainability:** detection, depth, geometry, risk, speech and navigation should be separate modules with tests.
- **Usability and Accessibility:** messages should be short and consistent; the interface should work with voice and large controls.
- **Safety:** messages should state approximate distances, never claim that a path is safe, and the interface should carry a prototype notice.
- **Privacy:** camera frames should be processed locally and the user's location should not be exposed by the status endpoints.
- **Security:** inputs such as locations, uploads and routes should be validated; the server should run on a trusted network.
- **Portability:** the same engine should serve a browser console and an Android client.

Chapter 4

SYSTEM DESIGN AND ANALYSIS

4.1 High Level Design

Figure 4.1 shows the high-level design as five layers. The input layer accepts frames from an IP or USB webcam, an uploaded video or a recorded test clip. The perception layer runs YOLO11s detection, ByteTrack tracking and Depth Anything V2 depth estimation. The spatial reasoning layer projects the depth map into 3D, estimates the ground plane with RANSAC, corrects the depth scale using the camera height and analyses the walking corridor. The risk and decision layer applies persistence, approach speed, time-to-collision, corridor membership, priority ranking and cooldowns. The interaction layer presents the annotated video, path status, distance and direction, captions and voice guidance.

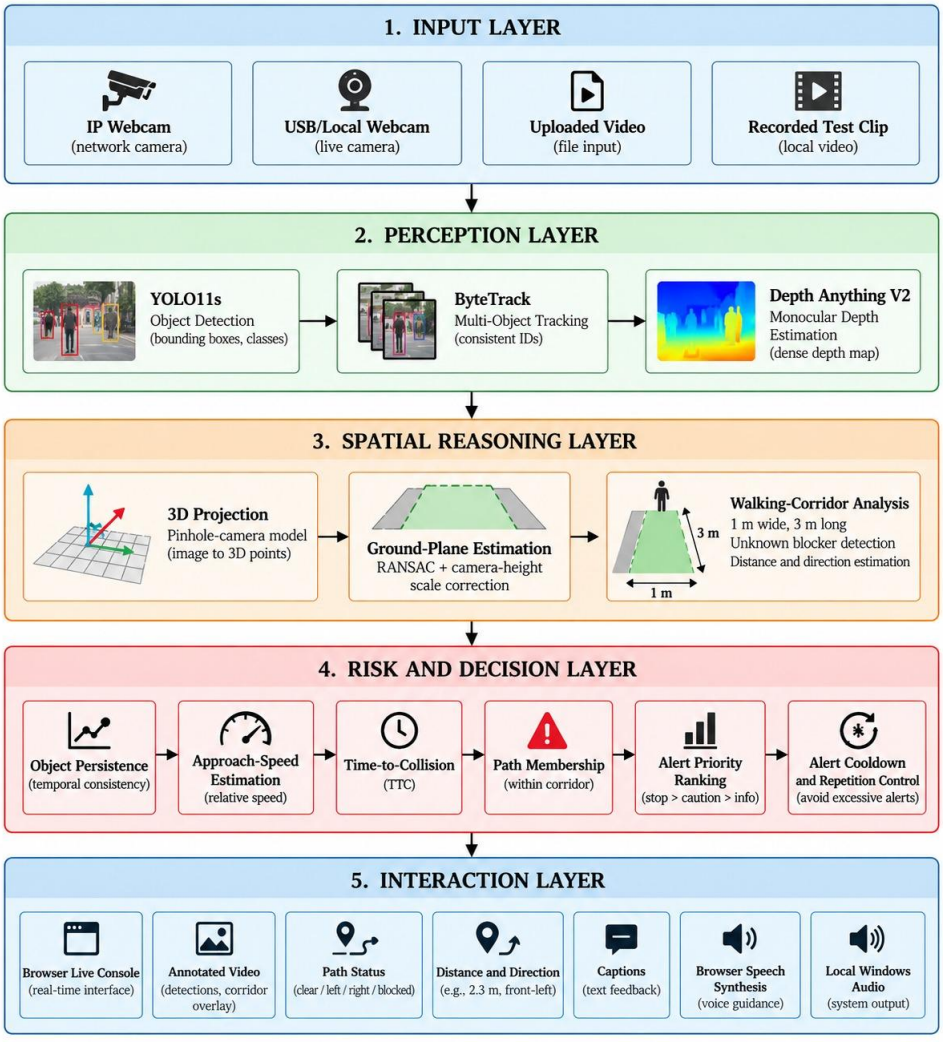


Fig 4.1: High level design – layered architecture of Safe-Stride Assist

Figure 4.2 shows the same system as components. Two clients, the Android app and the

browser console, talk to a single Python web server. The server hands frames to the inference engine, which runs the four processing stages, and passes GPS positions and route requests to the navigation manager. Outputs are the annotated frame with a state record, and voice with a stereo beep from the side of the obstacle. Place search and routing use public OpenStreetMap-based services.

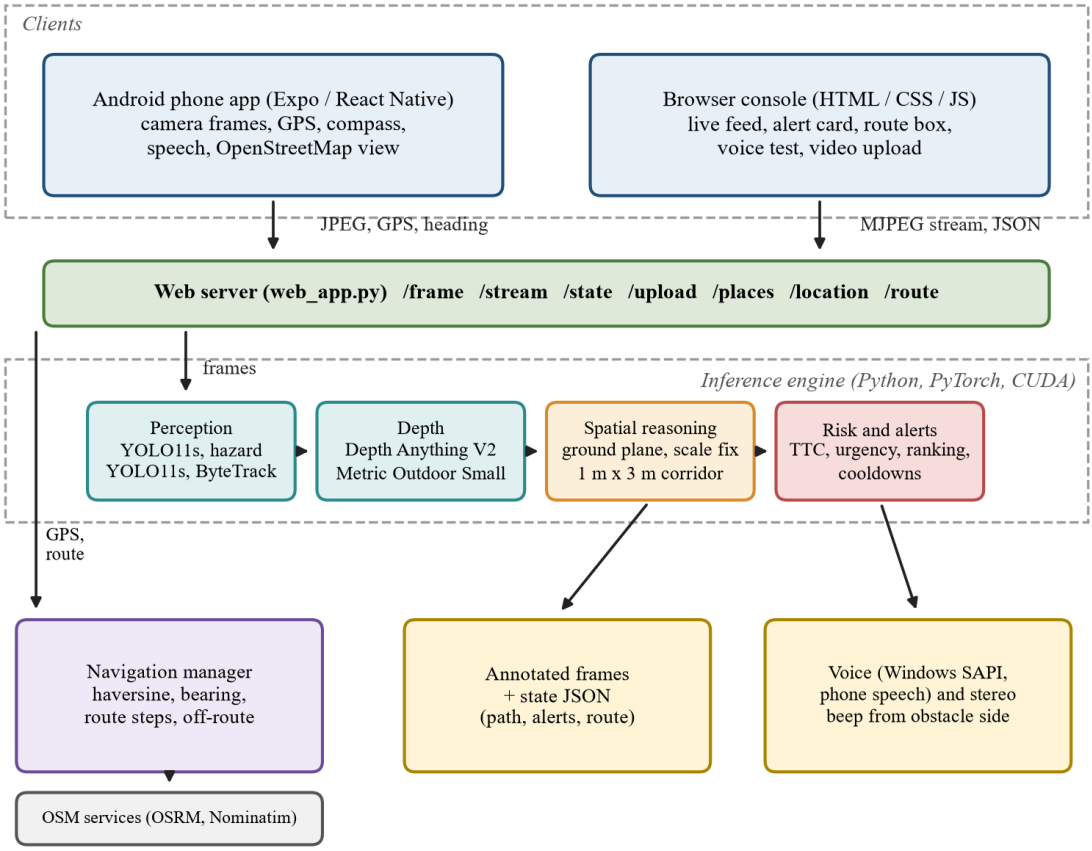


Fig 4.2: System components and their connections

4.2 Low Level Design

4.2.1 Use Case Diagrams

The walking-assistance module (Figure 4.3) lets the pedestrian start and stop monitoring, view the annotated feed, receive obstacle and stairs alerts, hear directional beeps, upload a video for analysis and test the voice. The camera or video source supplies the frames used by the alert and upload functions.

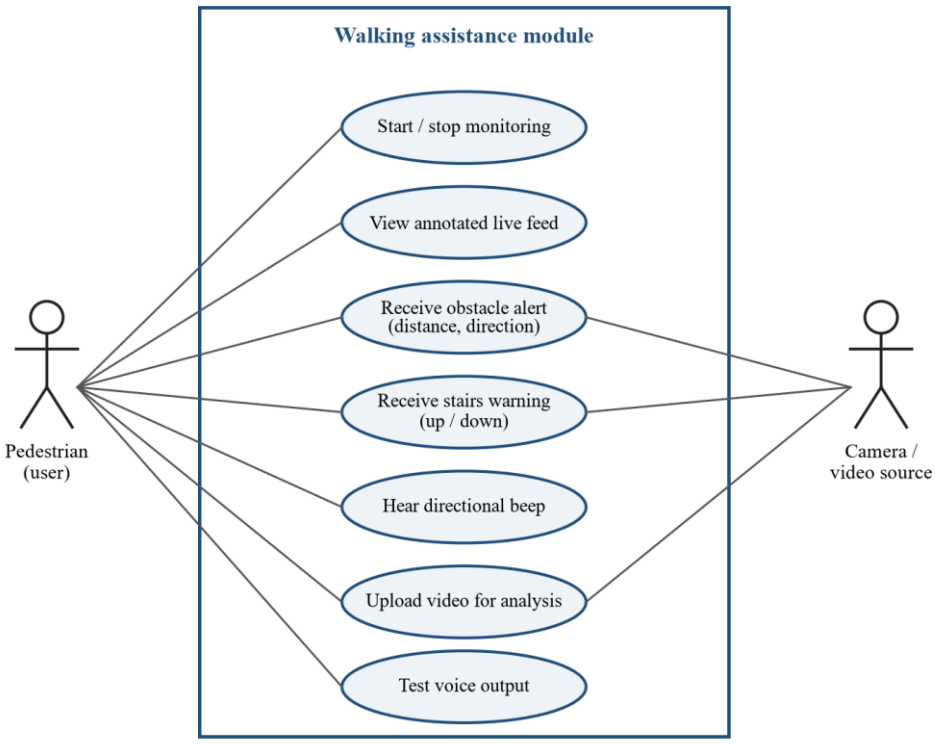


Fig 4.3: Use case diagram of the walking assistance module

The route-guidance module (Figure 4.4) lets the pedestrian search a destination, choose a suggestion and create a walking route. OpenStreetMap services provide the place search and the route. The module then gives distance and turn guidance, asks the user to align the phone with the road, warns when the user leaves the route and announces arrival.

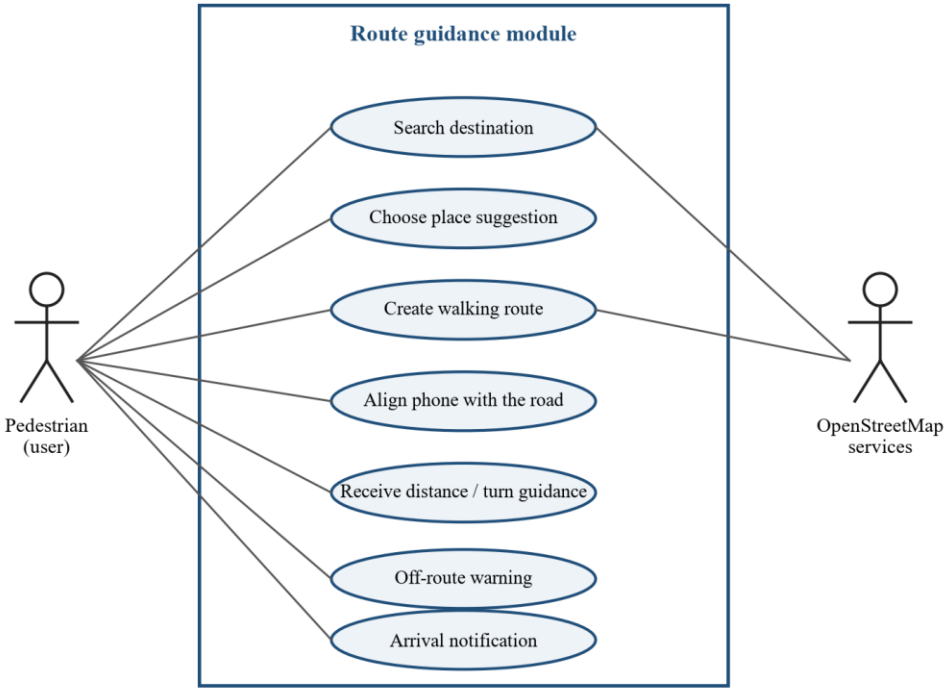


Fig 4.4: Use case diagram of the route guidance module

4.2.2 Data Flow Diagram

Figure 4.5 shows the data flow. A frame is decoded (1.0), objects are detected and tracked (2.0) and a depth map is estimated (3.0). Spatial analysis (4.0) combines boxes and depth into a path status. Alerts are ranked and announced (5.0) and rendered as output for the user (7.0). In parallel, route guidance (6.0) turns GPS, heading and the destination into guidance text with the help of the map services. Model weights, uploaded videos and the in-memory session state are the data stores.

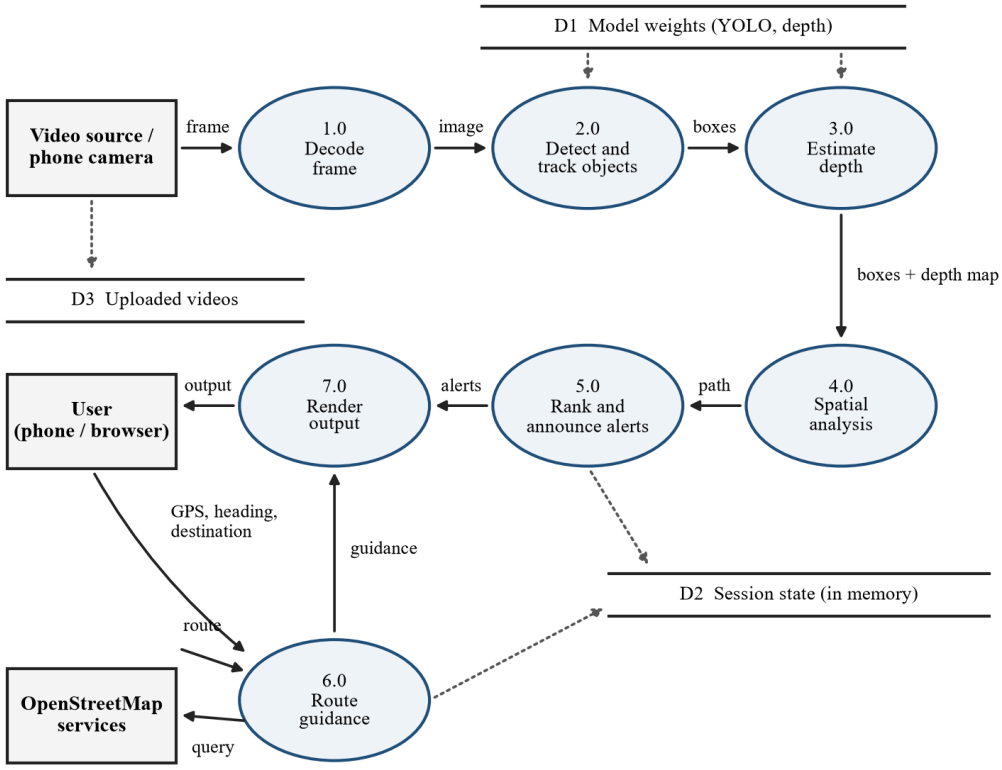


Fig 4.5: Data flow diagram

4.2.3 Sequence Diagram

Figure 4.6 shows what happens for one frame. The phone posts a JPEG to the server, which calls the inference engine. The engine calls the detector, the depth estimator, the spatial analyzer and the announcer in turn. If an alert is chosen, the speaker plays it on a separate worker thread, so the video loop never waits for audio. The annotated frame and the state return to the phone. Once per second the phone also posts its GPS position, and the server returns the route state and guidance text.

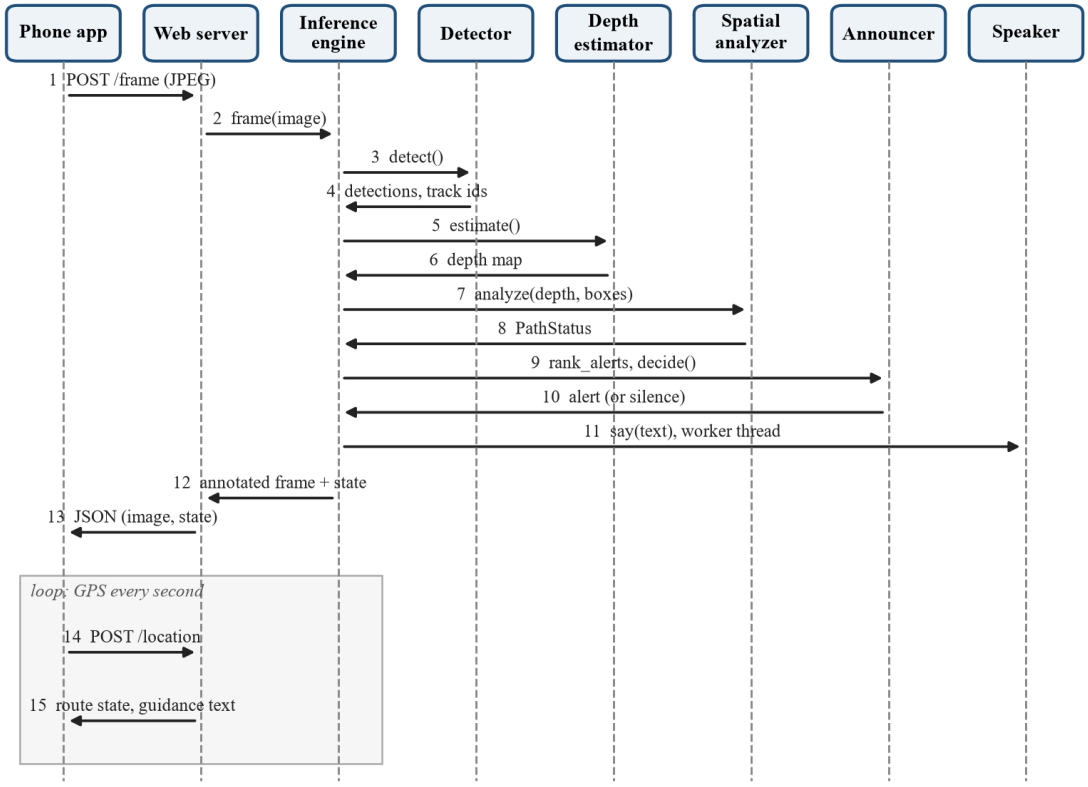


Fig 4.6: Sequence diagram for one frame and one GPS update

4.2.4 Walking Corridor Geometry

The walking corridor is a region on the ground in front of the camera, 1 m wide and 3 m long (Figure 4.7). A point belongs to a blocker when it lies inside the corridor and between 0.25 m and 2.0 m above the ground, so curbs are ignored but head-level obstacles count. Objects outside the corridor do not trigger path warnings.

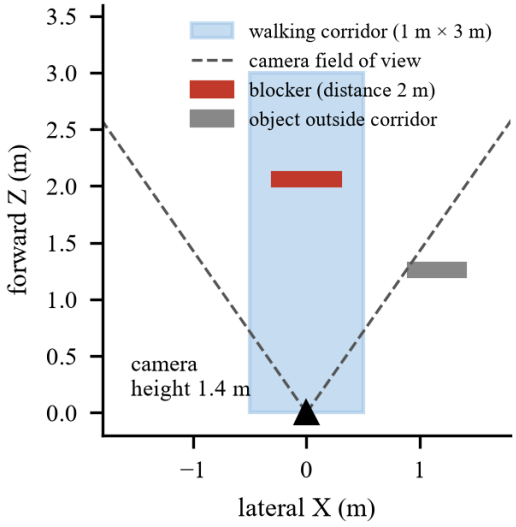


Fig 4.7: Walking corridor in camera coordinates

4.2.5 Navigation State Diagram

Route guidance follows the states in Figure 4.8. After a route is set, the system first asks the user to face the road (within 30° of the road bearing), then counts down the distance to the next maneuver, switches to the next instruction when the maneuver point is reached, and latches the arrived state. A new route is requested when two consecutive GPS readings are more than 50 m from the route.

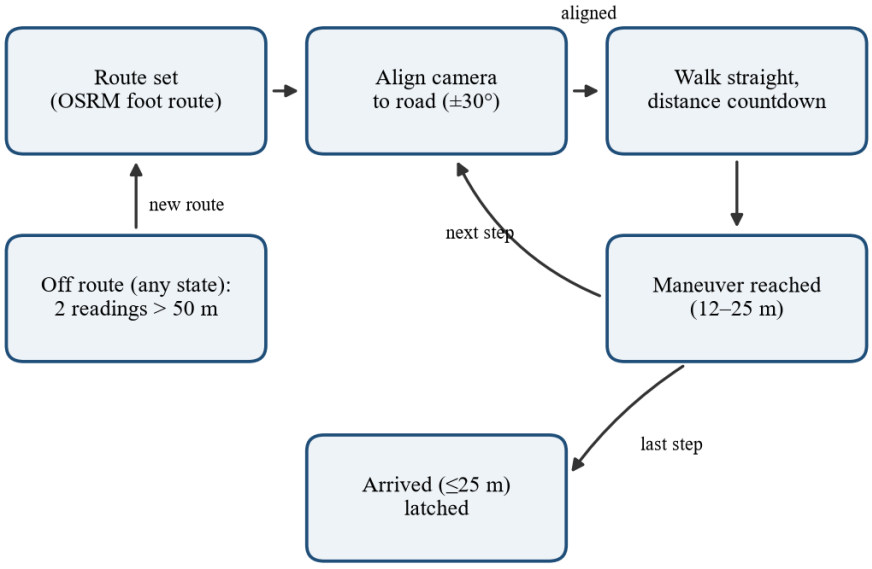


Fig 4.8: State diagram of the route guidance module

4.3 Architecture Overview

The project "Safe-Stride Assist – An AI-Based Safe Walking Assistance System for People with Visual Impairments" aims to give a walking person short, reliable guidance from a camera. The architecture separates learned perception from deterministic safety reasoning, so that each part can be tested on its own.

The **client layer** has two parts. The browser console is written in HTML, CSS and JavaScript; it shows the annotated live feed, the current priority message, a route box and diagnostics, and it speaks alerts with the browser speech engine. The Android client is written with Expo and React Native; it takes pictures with the camera, reads GPS and compass sensors, posts them to the server, shows the annotated image and speaks alerts and route instructions with a priority queue.

The **perception layer** runs YOLO11s for common objects, a second YOLO11s model for street hazards, ByteTrack for identities and Depth Anything V2 for depth. The **spatial reasoning layer** converts depth to 3D points, fits the ground with RANSAC, converts depth to

approximate metres and tests the corridor. The **decision layer** estimates time-to-collision from looming, computes urgency, ranks alerts and decides when to speak. The **navigation layer** turns GPS and heading into route guidance using OpenStreetMap routing.

The workflow begins when a frame reaches the server. The detectors and the depth model produce boxes and a depth map; the spatial analyzer produces a path status; the risk module adds approach information; the announcer selects at most one message; and the output layer draws the annotated frame and plays the voice. Because the modules are independent, the detector, the depth model or the speech engine can be replaced without changing the rest of the system.

Chapter 5

IMPLEMENTATION

5.1 Implementation

The system is implemented in Python (back-end and models) and TypeScript (Android client). Table 5.1 lists the main source files and what each one does. The sections that follow give the pseudocode of each module.

Table 5.1: Main source files

File	Purpose
src/blindnav/detector.py	YOLO detection, tracking and per-class thresholds
src/blindnav/depth.py	Depth Anything V2 depth map and object distance
src/blindnav/spatial.py	3D projection, ground plane, scale correction, corridor and stairs direction
src/blindnav/risk.py	Looming-based time-to-collision and approach speed
src/blindnav/announcer.py	Alert ranking, persistence, cooldowns and message wording
src/blindnav/speech.py	Text-to-speech, stereo beeps, live playback and soundtrack muxing
src/blindnav/navigation.py	Route state, heading alignment, off-route and arrival
src/blindnav/visualize.py	Overlays: corridor, boxes, trails, captions
web_app.py	Inference engine and HTTP server
web/ (index.html, app.js, styles.css)	Browser console
mobile/App.tsx and helpers	Android client (camera, GPS, compass, speech, map)
notebooks/train_hazards.ipynb	Training of the hazard detector
scripts/	Video runner, benchmark, evaluation and experiment scripts
tests/	41 automated tests

5.1.1 Object Detection and Tracking (detector.py)

Pseudocode:

```
BEGIN
    DEFINE class Detector(weights, conf, class_conf, id_offset)
        LOAD the YOLO weights
        SET model_conf = minimum of conf and all per-class thresholds
        SET class filter = walking-relevant classes (person, car, bus, bicycle, bench, ...)
    DEFINE detect(frame)
        RUN model.track(frame, conf = model_conf, classes = filter, FP16 on GPU, ByteTrack, persist)
        FOR each box in the result DO
            name = class name of the box
            IF confidence < class_conf[name] (or conf when no entry) THEN skip the box
            APPEND Detection(box, name, confidence, track id + id_offset)
        END FOR
```

```
        RETURN detections
    DEFINE reset_tracking()
        DROP the predictor so that a new video starts with a new tracker
    CREATE one Detector for COCO classes and one for the hazard model (id_offset = 100000)
END
```

5.1.2 Monocular Depth Estimation (depth.py)

Pseudocode:

```
BEGIN
    LOAD Depth Anything V2 Metric-Outdoor-Small in FP16 on the GPU (use the local cache
    if present)
    DEFINE estimate(frame)
        CONVERT the BGR frame to an RGB tensor on the GPU
        RESIZE so that the short side is 392 px and both sides are multiples of 14
        NORMALISE with the image mean and standard deviation
        RUN the network to obtain a low-resolution depth map
        UPSAMPLE the map to the frame size
        RETURN the depth map in metres (float32)
    DEFINE object_distance(depth, box)
        SHRINK the box by 25 % on every side to avoid background pixels
        RETURN the median depth inside the shrunk box
END
```

5.1.3 Ground Plane and Walking Corridor (spatial.py)

Pseudocode:

```
BEGIN
    f = (max(width, height) / 2) / tan(field_of_view / 2)
    DEFINE backproject(depth, f)
        FOR each sampled pixel (u, v) DO
            X = (u - W/2) * Z / f ;  Y = (v - H/2) * Z / f ;  Z = depth(u, v)
        END FOR
    DEFINE fit_plane_ransac(points)
        REPEAT 100 times
            PICK three random points and compute the plane normal n
            FLIP n so that it points down ; IF n_y < 0.8 THEN skip (not horizontal enough)
            COUNT points within tolerance 0.03 * median(Z) of the plane
        KEEP the plane with most inliers
        RETURN plane and inlier ratio
    DEFINE GroundEstimator.update(points)
        TAKE the lower 40 % of the points
        IF the best plane has more than 25 % inliers THEN
            scale = camera_height / height_of_plane
            APPEND scale to the last 90 values ; scale = median of those values
    DEFINE analyze(depth, detections)
        points = backproject(depth) ; IF no ground plane is known THEN RETURN empty
        status
```

```
        FOR each detection DO
            distance = distance * scale ; clock = direction bucket of the box centre
            in_path = |lateral offset| < 0.8 m AND distance < 3 m
        height = -scale * (n . P + d)  for every point
        FOR each stairs box DO slope = least-squares slope of height versus depth
            IF slope > 0.08 THEN direction = up ELSE IF slope < -0.08 THEN direction =
            down
        corridor = |X| < 0.5 m AND 0 < Z < 3 m ;  obstacle = corridor AND 0.25 m < height <
        2.0 m
        IF at least 15 obstacle points THEN
            blocked = true ; distance = 5th percentile of Z ; clock = median column
            head_level = (lowest obstacle point is above 1.0 m)
            guidance = side with fewer obstacle points (left or right)
        RETURN PathStatus
END
```

5.1.4 Motion and Time-to-Collision (risk.py)

Pseudocode:

```
BEGIN
    DEFINE update(t, detections, path, image_height)
        FOR each tracked detection DO
            SKIP if the box is shorter than 40 px or touches the bottom of the image
            growth = least-squares slope of ln(box height) over the last 1 s (at least 5 samples)
            IF growth > 1 / 10 s THEN ttc = 1 / growth ; approach_speed = distance * growth
        END FOR
        IF the path is blocked THEN
            closing_speed = - slope of the path distance over the last 1 s
            IF closing_speed > 0.3 m/s THEN path.ttc = distance / closing_speed
        ELSE forget the path history
        FORGET objects not seen for 2 s
END
```

5.1.5 Alert Ranking and Announcement (announcer.py)

Pseudocode:

```
BEGIN
    DEFINE urgency(distance, ttc, in_path, weight, range)
        proximity = clip(1 - distance / range) IF in_path ELSE 0
        time = clip(1 - ttc / 6 s) IF ttc exists AND distance < 12 m ELSE 0
        IF NOT in_path THEN time = 0.6 * time
        RETURN weight * max(proximity, time)
    DEFINE rank_alerts(detections, path)
        FOR each detection DO
            SKIP doors unless in the path and closer than 3 m ; SKIP non-finite or far distances
            label = stairs_up / stairs_down for confirmed stairs ; SKIP untracked objects
            outside the path
            CREATE an Alert with urgency, clock position and path flag
```

```
        IF the path is blocked THEN
            named = nearest named hazard to the depth obstacle
            IF named produced an alert THEN attach the clearer side to it
            ELSE (or head-level obstacle) ADD a generic "obstacle" alert
        level = URGENT IF in_path OR urgency >= 0.6 ; WARNING IF urgency >= 0.3
        RETURN alerts sorted by (urgent, stairs in path, in path, 11 o'clock, urgency)
    DEFINE Announcer.decide(t, alerts)
        UPDATE first_seen and last_present ; FORGET alerts absent for 3 s
        FOR each alert in order DO
            IF NOT in_path AND seen for less than 0.2 s THEN skip
            IF NOT in_path AND NOT urgent AND last message was less than 0.1 s ago
            THEN skip
            IF never spoken OR level increased THEN SPEAK it
            ELSE IF 5 s have passed AND it is at least 1 m closer THEN SPEAK it
        RETURN the alert to speak, or none
END
```

5.1.6 Speech and Audio Output (speech.py)

Pseudocode:

```
BEGIN
    DEFINE Synthesizer(text)
        SPEAK the text with Windows SAPI into a 22 kHz WAV file ; READ the samples
        TRIM the silence around the words ; CACHE the result for repeated messages
    DEFINE render(alert, text)
        beep = 1200 Hz double beep IF urgent ELSE 700 Hz single beep
        PAN the beep fully toward the obstacle's clock position ; PAN the voice only 30 %
        RETURN beep + short pause + voice
    DEFINE LiveSpeaker.say(alert, text)
        LOCK ; IF alert is URGENT THEN generation = generation + 1 ; FLUSH the queue
        PUT (generation, alert, text) on the queue
    WORKER THREAD (the only thread that touches the audio device)
        INITIALISE COM ; CREATE two voices
        LOOP : GET an item ; SKIP it if its generation is old
            OPEN an output stream
            FOR each chunk of 1024 frames DO
                IF the generation changed THEN stop playing this clip
                WRITE the chunk
    DEFINE SoundtrackRecorder.mux(video, out)
        PLACE each clip at its video time without overlap ; WRITE a WAV file
        RUN ffmpeg to encode H.264 video with AAC audio
END
```

5.1.7 Route Guidance (navigation.py)

Pseudocode:

```
BEGIN
    DEFINE set_route(route, destination)
```

STORE the maneuver steps and the route geometry ; RESET step, arrival and message state
DEFINE update(latitude, longitude, heading, accuracy, heading_timestamp)
heading is usable only if it is less than 5 s old
IF arrived already THEN RETURN "arrived" without a new message
IF distance to destination <= 25 m THEN latch arrived ; SPEAK "You have arrived"
deviation, remaining = nearest point on the route polyline
IF deviation > max(50 m, 1.5 * accuracy) THEN count = count + 1
IF count >= 2 THEN SPEAK "You are off the planned route"
WHILE the next maneuver is reached (within 12-25 m or passed along the segment) DO step = step + 1
target = bearing after the current maneuver
IF heading unknown THEN SPEAK "Camera direction unavailable. Stop ..."
delta = difference between heading and target
IF |delta| > 30 degrees THEN SPEAK "Turn left / right to face the road" (or "Turn around" above 150)
ELSE distance = distance to the next maneuver
SPEAK "Walk straight for N meters, then <next instruction>"
SPEAK again only if the instruction changed or 8 s have passed
END

5.1.8 Inference Engine and Web Server (web_app.py)

Pseudocode:

BEGIN
DEFINE InferenceEngine.frame(image, speak, timestamp)
detections = COCO detector + hazard detector
IF depth is due (every 2nd frame, or the frame size changed) THEN depth = estimate(image)
FOR each detection DO distance = object_distance(depth, box)
path = analyze(depth, detections) ; KEEP stairs only when up/down is confirmed
UPDATE motion (time-to-collision) ; alerts = rank_alerts(detections, path)
LATCH the safety state until 3 clear frames
alert = announcer.decide(time, alerts)
IF alert THEN text = message(alert) ; SPEAK it (browser and phone speak for themselves)
ELSE IF the clear-side guidance changed THEN SPEAK "Keep left / right"
DRAW trails, path, boxes and FPS ; UPDATE the shared state under a lock
RETURN the annotated image
HTTP SERVER
GET /stream : reset the engine, read the source, send MJPEG frames (video time from frame number)
POST /frame : decode JPEG or base64 JSON, run the engine under a lock, return image + state
POST /location : validate latitude, longitude and numbers, update navigation
POST /route : geocode or use the chosen place, request a foot route, set the route
GET /places : place suggestions ; GET /state : public state without the GPS position
POST /upload : save a video file (extension and size checked)

END

5.1.9 Hazard Model Training (train_hazards.ipynb)

Pseudocode:

BEGIN

DOWNLOAD the street dataset (22 classes, polygon labels)
KEEP six classes: stairs, pole, pillar, bin, cone, obstacle ; DROP the tree class (too few examples)
CONVERT polygon labels to boxes ; ADD background images to the training split
CHECK a sample of boxes visually
TRAIN YOLO11s from COCO weights : 60 epochs, batch 32, 640 px, patience 15, seed 0
EVALUATE on the held-out test split : precision, recall, mAP50, mAP50-95 per class
SET per-class thresholds : obstacle 0.5, bin disabled
SAVE the best weights as hazards.pt

END

5.1.10 Android Client (mobile/App.tsx)

Pseudocode:

BEGIN

ASK for camera and location permission ; IF denied THEN stop and show the reason
START the GPS watcher (every 1 s), the heading watcher (magnetic declination) and the sensors
camera_bearing = bearing of the camera axis from gravity, magnetic field and declination
LOOP while monitoring
    TAKE a picture (JPEG, base64) ; POST /frame ; SHOW the annotated image
    UPDATE status, depth flag and FPS ; IF a new safety alert exists THEN SPEAK it with high priority
ON each GPS update
    POST /location with the heading age ; GET /state ; APPLY the navigation guidance
    SPEAK navigation text unless a safety alert is active (safety prompts pre-empt it)
ON route request
    POST /route with the selected place and its coordinates ; SHOW the route on the map

END

5.2 Key Parameters

Table 5.2 lists the main constants of the system. They are defined at the top of the corresponding source files, so they can be changed without touching the logic.

Table 5.2: Key parameters

Parameter	Value	Used for
Camera height	1.4 m	Converts the ground plane into a depth scale
Field of view	70° (longer side)	Focal length for back-projection
Corridor	1 m wide, 3 m long	Region tested for blockers

Parameter	Value	Used for
Blocker height	0.25 m to 2.0 m	Ignores curbs, keeps head-level objects
Head-level limit	1.0 m	Obstacle with nothing below it
Minimum blocker points	15	Rejects isolated noise
RANSAC	100 iterations, tolerance 3 % of median depth	Ground-plane fit
Scale history	90 frames (about 3 s), median	Stable depth scale
Depth input / refresh	392 px, every 2nd frame	Speed and detail trade-off
Detector confidence	0.35 (stairs 0.10, obstacle 0.5, bin disabled)	False-alarm control
TTC window	1 s, at least 5 samples	Looming estimate
Urgency thresholds	warning 0.3, urgent 0.6	Alert level
Persistence / repeat	0.2 s / after 5 s and 1 m closer	Avoids repeated speech
Route thresholds	off-route 50 m, arrival 25 m, alignment 30°	Route guidance
Heading freshness	5 s	Ignores stale compass data

5.3 User Interfaces

The browser console (Figure 7.7) is served by the same Python server. It shows the annotated live feed, the current priority message, objects, inference speed and depth status, a route box and a legend. Buttons start and stop monitoring, test the voice and upload a video. The Android client has the same functions plus a live OpenStreetMap view, GPS and compass status, and a priority speech queue in which safety alerts interrupt navigation messages. Table 5.3 lists the server endpoints.

Table 5.3: Server endpoints

Endpoint	Method	Purpose
/stream	GET	MJPEG stream of the annotated feed (webcam or uploaded video)
/frame	POST	Analyse one frame sent by the phone; returns image and state
/state, /health	GET	Public state (alerts, FPS, navigation); the GPS position is not included
/location	POST	GPS position, accuracy, heading and its age
/places	GET	Destination suggestions (Photon or Google Places)
/route	POST	Create a walking route to a chosen place
/upload	POST	Upload a video (mp4, mov, avi, mkv, webm, m4v; up to 2 GB)

Chapter 6

TESTING

6.1 Types of Testing

Testing is an essential phase of Safe-Stride Assist because the system gives advice to a walking person. The following types of testing were applied. Results that were measured are reported in Chapter 7; nothing is claimed that was not run.

1. Unit Testing: Individual functions are tested with small inputs: stairs direction from a synthetic ramp, alert ranking and repetition rules, navigation messages and distances, speech interruption logic and input validation of the server.

2. Integration Testing: The inference engine is run with stub detectors and a counting depth model to check how depth refresh, resizing and state updates work together. The HTTP server is started on a local port and called with real requests.

3. System Testing: The complete system was run on recorded walking videos with the real models (detectors, depth, geometry, alerts) and the browser console was driven through a real session.

4. Functional Testing: Scenario tests check that a blocker in the corridor is found at the right distance, that an object outside the corridor is ignored, that the clear side is suggested, and that route messages follow the route.

5. AI Model Testing: The hazard detector was evaluated on 540 held-out images with precision, recall, mAP50 and mAP50-95 per class. The depth scale correction and the time-to-collision estimate were tested on synthetic scenes with known geometry.

6. Performance Testing: Per-stage time per frame was measured with and without refreshing depth on every frame, and the full pipeline speed was measured on real video.

7. Compatibility Testing: The browser console was exercised in the integrated browser of the development environment, and the server with the phone client's request formats. Other browsers and phones were not tested.

8. Security Testing: Invalid and malformed inputs (coordinates out of range, non-numeric values, missing fields) are rejected with an error, and the status endpoints do not return the user's position. No penetration test was carried out.

9. Regression Testing: All 41 automated tests are re-run after every change; defects found during verification were each given a test so that they cannot return.

These activities give a structured view of correctness, integration and performance. They do not replace testing with visually impaired users, which has not been carried out.

6.2 Test Cases

Module 1: Walking corridor, stairs and alerts (scenario tests)

Table 6.1: Scenario test cases (expected versus obtained)

No	Scenario	Expected	Obtained	Status
1	Surface rising away from user (slope +0.5)	stairs up	stairs up	Pass
2	Surface falling away from user (slope -0.5)	stairs down	stairs down	Pass
3	Flat ground inside the stairs box	no direction	no direction	Pass
4	Obstacle 2.0 m ahead, depth scale ×2	blocked, 2.0 ± 0.3 m	blocked, 2.0 ± 0.3 m (1.95 m)	Pass
5	Obstacle 1.2 m to the side (outside corridor)	path clear	path clear	Pass
6	Obstacle slightly right of centre	keep left	keep left	Pass
7	Detection with undefined distance	no alert	no alert	Pass
8	Same obstacle returns after path cleared	announced again	announced again	Pass
9	GPS 150 m off the route (two readings)	off-route warning	off-route warning	Pass
10	Arrival, then GPS jitter 40 m away	arrival announced once	arrival announced once	Pass

Module 2: Route guidance (automated unit tests)

Table 6.2: Route guidance tests

No	Test	Expected result	Status
1	Position between route vertices is on the route and uses route distance	On route; distance remaining about 111 m	Pass
2	Snapshot includes the route geometry for the map	Geometry points returned	Pass
3	Off-route needs two consecutive GPS readings	Warning only on the second reading	Pass
4	Off-route threshold accounts for GPS accuracy	No warning when the error is within accuracy	Pass
5	Departure corrects compass, then guides along the first road	“You are facing the mapped road … walk straight for 75 meters, then turn right”	Pass
6	Stale camera direction gives no walk instruction	“Camera direction unavailable. Stop …”	Pass
7	Phone facing east on a northbound departure	“Turn left to face the road”	Pass
8	Nearly opposite heading	“Turn around to face the road”	Pass

No	Test	Expected result	Status
9	Turn is not skipped; final leg stays on the last segment	Step 1 reached; message for the next road	Pass
10	Turn advances when a GPS sample jumps past the waypoint	Step advances to the turn	Pass
11	Arrival is announced once and not repeated	One message, then silence	Pass
12	Arrival stays latched under GPS jitter	Still arrived; no new message	Pass
13	Walk instruction counts down the distance to the next turn	75 m, then 25 m after walking	Pass
14	A shrinking distance alone is not re-spoken early	No repeat before the timer	Pass

Module 3: Alerts, geometry, speech and server (automated unit tests)

Table 6.3: Alert, geometry, speech and server tests

No	Module	Test	Expected result	Status
1	Announcer	A new obstacle in the same place is announced again after the path cleared	Announced again	Pass
2	Announcer	The same obstacle is not repeated while it stays	Silent	Pass
3	Announcer	A detection with an undefined distance is ignored	No alert, no crash	Pass
4	Announcer	A named hazard without its own alert does not hide a blocked path	Generic obstacle alert kept	Pass
5	Spatial	Surface rising away from the user	Stairs up	Pass
6	Spatial	Surface falling away from the user	Stairs down	Pass
7	Spatial	Flat ground inside the stairs box	No direction	Pass
8	Speech	A warning neither interrupts nor flushes	Queue unchanged	Pass
9	Speech	An urgent alert drops queued alerts	Queue holds only the urgent alert	Pass
10	Speech	A playing clip is abandoned when an urgent alert arrives	Playback stops within one chunk	Pass
11	Speech	A clip plays completely without interruption	All chunks written	Pass
12	Server	Depth is recomputed on every frame when the interval is 1	4 calls for 4 frames	Pass
13	Server	Depth is reused between refreshes	2 calls for 6 frames	Pass
14	Server	Depth is recomputed when the frame size changes	2 calls	Pass
15	Server	Number parsing accepts numbers and rejects null, booleans, strings, NaN and infinity (7 cases)	Float or none	Pass
16	Server	A location with null accuracy is accepted and not exposed in /state or /health	200; no location in output	Pass
17	Server	Invalid locations (out of range, non-numeric, missing) (4 cases)	HTTP 400	Pass
18	Server	A non-numeric heading timestamp does not crash navigation	HTTP 200	Pass

All 41 automated tests passed at the time of writing (the rows above cover them; two rows group several parametrised cases).

6.3 Defects Found and Fixed During Testing

Writing the tests and running the system exposed real defects. Table 6.4 lists them with the fix. Each defect has a test or a measurement that fails without the fix.

Table 6.4: Defects found and fixed

No	Module	Defect found	Fix
1	spatial	Stairs up and down labels were inverted (confirmed on a synthetic ramp)	Sign of the depth slope corrected
2	announcer	An obstacle that cleared and returned was never announced again	Alerts absent for 3 s are forgotten
3	announcer	A NaN distance produced a phantom urgent alert and could crash the message code	Non-finite distances skipped
4	announcer	A named hazard that made no alert hid the generic obstacle alert, so a blocked path stayed silent	Generic alert kept unless the named hazard has its own alert
5	detector	Per-class thresholds below the global 0.35 (for example stairs 0.10) never took effect	Model runs at the lowest threshold; per-class filter does the rest (side effect: with the stairs-only weights the 0.10 stairs threshold now produces many stairs detections, see Section 7.7)
6	web_app	With depth refresh 1 the depth froze after the first frame; depth was reused after a frame-size change	Refresh test corrected; size change forces a new depth map
7	web_app	Model paths depended on the working directory, silently disabling the hazard model	Paths resolved from the project folder
8	web_app	Uploaded videos used wall-clock time; tracker and announcer state leaked between videos	Video time used; engine reset at each stream
9	web_app	A null GPS accuracy gave an error; a bad heading timestamp gave a server error	Values validated with a finite-number check
10	web_app	/state and /health exposed the user's GPS position	Location removed from public output
11	web_app	Windows connection-abort tracebacks on client disconnect	Treated as a normal disconnect
12	speech	Stopping audio from the request thread while the worker started a clip crashed the whole process (reproduced in 2 of 3 stress runs)	Only the worker touches audio; urgent alerts bump a generation counter (6 of 6 stress runs survive)
13	speech	A missing speech engine killed the audio thread silently; temporary files collided between processes	Error reported once; per-process temporary files

No	Module	Defect found	Fix
14	navigation	“Arrived” repeated every 8 s and could revert to walking guidance under GPS jitter	Arrival announced once and latched
15	navigation	The walking distance never counted down, so guidance sounded stuck	Distance to the next maneuver is spoken
16	mobile	Monitoring stayed “running” when location permission failed; compass declination ignored when uncalibrated; GPS watcher not started after a route	Monitoring stops with the reason; declination accepted; watcher started with the route

Chapter 7

RESULTS AND DISCUSSIONS

This chapter reports what was measured by running the project code. Controlled experiments use synthetic scenes with exactly known geometry; the real-video runs have no hand-labelled ground truth. The two are kept separate throughout.

7.1 Hazard Detector Results

The street-hazard detector is a YOLO11s model trained for 60 epochs on six classes (Table 7.1). Figure 7.1 shows the training and validation curves recorded during training; the metrics were still improving slowly at the end. On the held-out test split (540 images, 536 instances) it reached precision 0.667, recall 0.584, mAP50 0.556 and mAP50-95 0.401, at 11.4 ms per image on a Tesla T4. The validation split gave a higher mAP50 of 0.684, so the model generalises less well to unseen images than validation suggests.

Table 7.1: Hazard dataset after filtering (instances per split) and test results

Class	Train	Valid	Test	P	R	mAP50	mAP50-95
stairs	183	19	14	0.660	0.555	0.571	0.437
pole	1637	107	92	0.570	0.739	0.585	0.418
pillar	510	193	175	0.728	0.680	0.746	0.589
bin	36	25	38	0.652	0.026	0.034	0.029
cone	123	32	74	0.972	0.944	0.942	0.707
obstacle	234	10	143	0.419	0.559	0.458	0.228
all			536	0.667	0.584	0.556	0.401

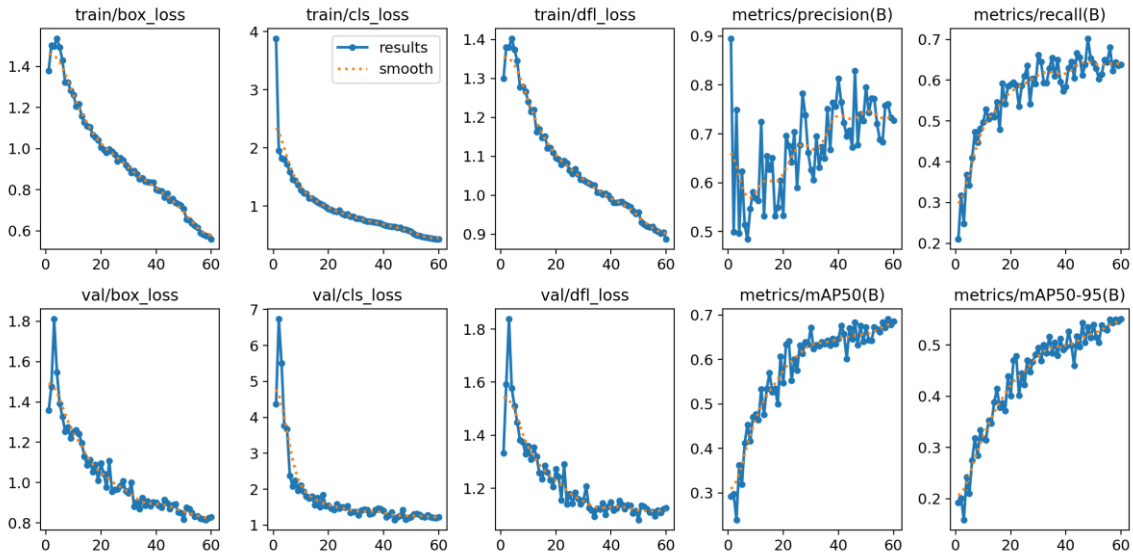


Fig 7.1: Training and validation curves of the hazard detector (from the executed notebook)

Cones and pillars are detected well, poles reasonably (Figure 7.3). Bins are almost never found (recall 0.026) because the training set held only 36 bin instances, so the live system disables that class. The "obstacle" class has low precision, so it needs a confidence of 0.5. The normalised confusion matrix (Figure 7.2) shows the same pattern: 92 % of real bins are missed and 66 % of "obstacle" predictions are false. The stairs estimate rests on only 14 test instances and should be read as indicative.



Fig 7.2: Normalised confusion matrix on the test split (from the executed notebook)

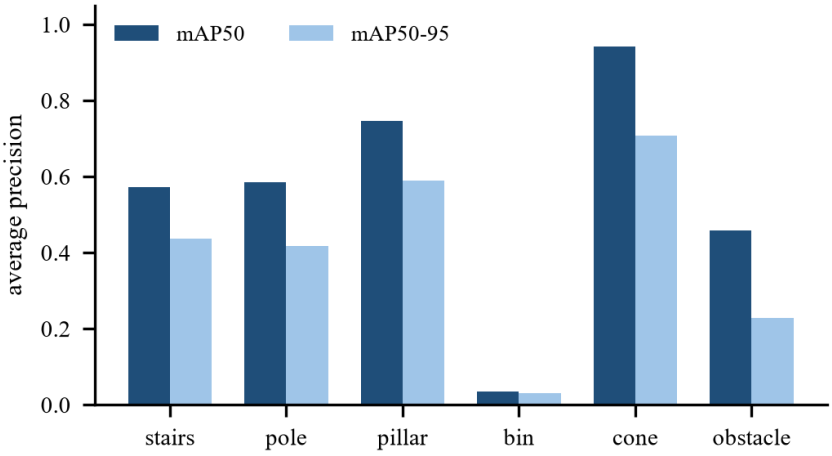


Fig 7.3: Per-class average precision on the test split

7.2 Effect of Ground-Plane Scale Correction

A synthetic scene with exact geometry (flat ground seen from 1.4 m, an obstacle 0.8 m wide and 1.2 m tall at 1.5 to 2.8 m, 2 % depth noise, five noise seeds) isolates the scale correction. The raw depth was multiplied by 1.0, 1.5, 2.0 and 3.0 to imitate a monocular network that is

wrong by a constant factor. Table 7.2 and Figure 7.4 show that with correction the mean distance error was 2.4 cm at every factor and every blocker was found. Without it the error grew with the scale error and, because the corridor is only 3 m long, more and more blockers were no longer seen at all. These scenes are an idealised upper bound: real scenes have uneven ground, occlusion and a camera height that changes.

Table 7.2: Distance error and detection with and without scale correction

Depth scale error	Error with correction (cm)	Detected (%)	Error without correction (m)	Detected without (%)
×1.0	2.4	100	0.08	100
×1.5	2.4	100	0.78	50
×2.0	2.4	100	1.38	25
×3.0	2.4	100	-	0

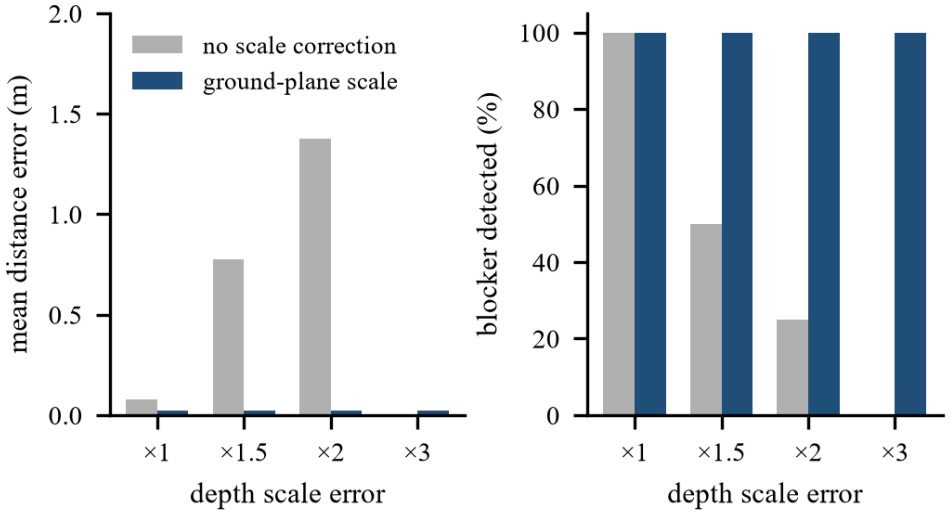


Fig 7.4: Distance error (left) and detection rate (right) with and without scale correction

7.3 Time-to-Collision Accuracy

The looming estimator was tested on a simulated 1.7 m tall person approaching a 1280×720 camera at constant speed from 10 s before impact, with 1.5 px of noise on both box edges, 30 frames per second and 20 noise seeds per speed. Table 7.3 lists the mean estimate at four true values. An estimate was available at every checkpoint. It was too large by 0.35–0.52 s at every speed, close to half of the one-second window used for the slope fit, because the fit reports the average over the last second. The alert therefore arrives about half a second late, which is 1.5 m of travel for a 3 m/s bicycle. Subtracting half the window would likely remove most of this bias, but this was not tested.

Table 7.3: Estimated time-to-collision (s) at four true values

Closing speed	True 6.0	True 4.0	True 3.0	True 2.0
1 m/s	6.42	4.50	3.48	2.48
1.5 m/s	6.36	4.50	3.48	2.48
3 m/s	6.35	4.52	3.48	2.49

7.4 Processing Speed

Table 7.4 gives the mean time per frame measured on a NVIDIA GeForce RTX 4060 Laptop GPU for a walking video (1280 px wide) with the COCO detector, the hazard detector, depth, spatial analysis and drawing. Refreshing depth on every second frame lowers the total from 62.2 ms (16.1 FPS) to 49.0 ms (20.4 FPS). Depth estimation is the largest cost. Risk analysis and the announcer take about 0.1 ms.

Table 7.4: Mean time per frame (ms)

Stage	Depth every frame	Depth every 2nd frame
detect (COCO)	12.3	12.9
detect (hazards)	10.9	11.3
depth	29.7	15.1
spatial	7.2	7.5
risk + announcer	0.1	0.1
drawing	1.9	2.1
TOTAL	62.2	49.0

7.5 Behaviour on Real Walking Video

The full pipeline was run on 5 clips (90 s in total): a street clip, a crowded station walkway, a city sidewalk with a construction barrier, an indoor lobby and a short indoor phone recording of a living room. There is no ground truth, so Table 7.5 reports what can be measured: speed (mean 14.6 FPS including all models), alert rate and the share of frames with a blocked path. The living-room clip is a failure case: the stairs detector fires on the patterned carpet (Figure 7.6) and the path is reported blocked in every frame, which gives 175 alerts per minute. The stairs model, trained on outdoor scenes, is not reliable on indoor floors, and the alert rate on the other clips (6 to 48 per minute) is more representative, although it too includes stairs alerts that were not checked (see Section 7.7). Whether individual alerts on the other clips were correct was not checked by hand. As a distance sanity check, the depth-based distance to fully visible people was compared with the distance implied by a 1.7 m person height: the ratio had a median of 1.08 (interquartile range 0.87–1.34, 12111 measurements). Both estimates rely on

assumptions, so this tests consistency, not accuracy.

Table 7.5: Full-pipeline behaviour on walking video

Clip	Length (s)	FPS	Alerts per min	Urgent	Blocked (%)
street	20	16.6	6.0	0	0
living room*	10	21.8	175.3	28	100
walk_001-1	20	14.5	48.0	8	2
walk_002-1	20	9.2	24.0	6	0
walk_003-2	20	10.9	15.0	5	39



Fig 7.5: Annotated frame (left) with distance, direction and time-to-collision (magenta) and the depth map (right) on the sidewalk clip

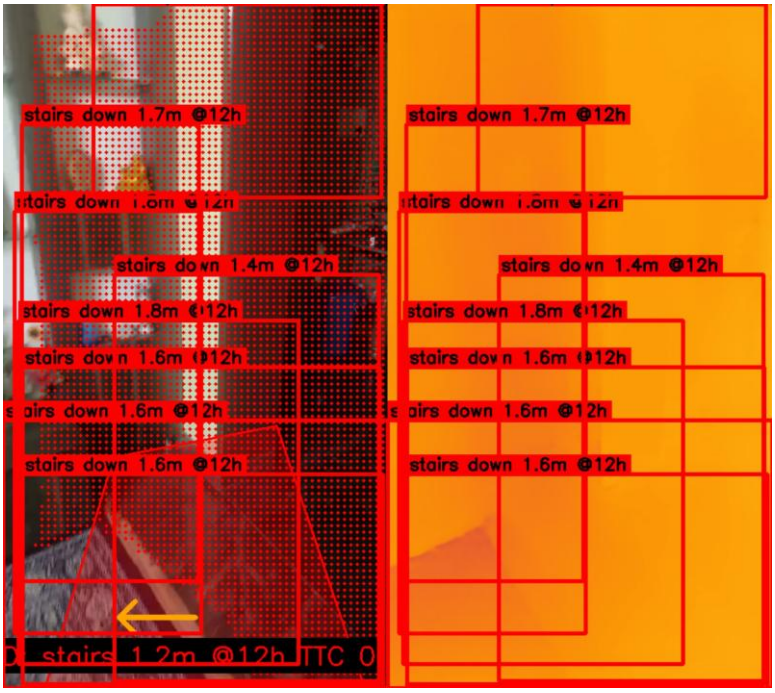


Fig 7.6: Failure case on the living-room clip: false stairs detections on a patterned carpet

7.6 Route Guidance under GPS Noise

A 155 m two-leg route was walked 20 times per noise level at 1.4 m/s with Gaussian GPS error, a compass error of 8° and one GPS sample per second (Table 7.6). Navigation reached the turn and the destination in all runs, the spoken distance decreased between the first and last

message in all runs, and no run raised a false off-route warning. About 8.7 messages per 100 m were spoken. These are simulations; the compass-to-heading conversion on a real phone has not been validated on a device.

Table 7.6: Route guidance simulation (20 runs per row)

GPS error σ (m)	Arrived (%)	Turn step (%)	False off-route (%)	Messages per 100 m
3	100	100	0	8.9
6	100	100	0	8.8
10	100	100	0	8.4

7.7 Overall System Output

The final system brings the functions together in a browser console (Figure 7.7) and an Android client. The console shows:

- the annotated live feed with boxes, distance, clock direction and time-to-collision;
- the corridor overlay and the path status (clear or blocked, with the clear side);
- the current priority message, which turns red for “Stop!” alerts and is spoken aloud;
- objects in view, inference speed, depth status and system status;
- a route box for destination search and route status.

Messages produced in the real-video runs include “Truck approaching at 11 o'clock, 3 meters”, “Stop! Path blocked, 3 meters. Keep Left.”, “Person approaching at 1 o'clock, 6 meters” and “Stop! bus approaching ahead, 3 meters”. Their correctness was not verified against ground truth. Figure 7.7 also shows a weakness: several “stairs up” boxes appear on an ordinary sidewalk. The stairs threshold of 0.10 was chosen for the six-class model, but the weights in the project's model folder are a single-class stairs model, and at that low threshold it fires often outdoors as well as on the indoor carpet. Stairs alerts should therefore be treated as unverified until the six-class weights are restored or the threshold is re-tuned.

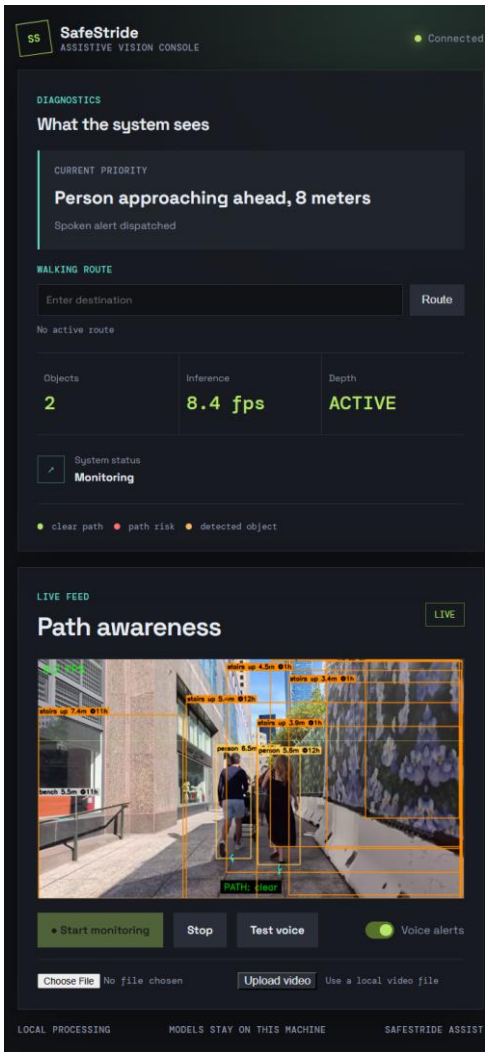


Fig 7.7: Browser console during a session on the sidewalk clip

7.8 Discussion

Three results shape the design. First, scale correction is what makes a 3 m corridor usable with a monocular network: without it the distance error grows with the scale error and blockers leave the corridor. Second, depth estimation is the largest cost (Table 7.4), so refreshing it every second frame buys about four frames per second at the price of slightly older geometry. Third, the hazard detector generalises less well than its validation score suggests, and the classes with few examples (stairs, bins) are the weakest, so rare but dangerous hazards need more data.

Verification also paid off. The explicit scenario tests and the stress runs found the defects listed in Table 6.4, including inverted stairs labels and a native crash in the audio path that no single screenshot would have shown. The main limitations are: monocular depth and a fixed camera height make distances approximate; the controlled experiments use idealised geometry; the real-video clips have no ground truth; the stairs detector produced false detections on an indoor patterned floor and on an ordinary sidewalk; the bin class is not usable; the weights in the

project's model folder are a single-class stairs model rather than the six-class model evaluated in Table 7.1; and no study with visually impaired users has been carried out. The prototype has not been validated for safety and must be used only together with a cane, a guide dog or human support.

7.9 Conclusion

Safe-Stride Assist shows that one camera can give path-aware, prioritised walking guidance when detection, tracking, scale-corrected monocular depth, corridor reasoning, risk ranking and route guidance are combined in one local system. It runs at about 20 frames per second on a laptop GPU. Ground-plane scale correction removed the large distance errors that a constant depth-scale error produces, the alert and route modules behaved consistently in simulation, and 41 automated tests protect the behaviour that was fixed during verification. The system is a research and teaching prototype, not a certified mobility device.

7.10 Future Scope

- **Evaluation:** collect labelled walking data and carry out a supervised study with visually impaired users.
- **Calibration:** estimate camera height and pitch from the phone sensors and calibrate the risk weights.
- **Hazard model:** retrain the bin and obstacle classes with more data and add curbs, potholes and open drains.
- **Time-to-collision:** remove the half-window lag of the looming estimate and test the correction.
- **Haptics:** add vibration cues for urgent alerts on the phone and wearables.
- **Voice control and emergency support:** voice commands, an SOS function and Kannada and Hindi speech.
- **Security:** add authentication to the server and automatic deletion of uploaded videos.
- **Devices:** validate the compass handling on several phones and port speech to non-Windows servers.

REFERENCES

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016. https://arxiv.org/abs/1506.02640

[2] Y. Zhang, P. Sun, Y. Jiang et al., "ByteTrack: Multi-object tracking by associating every detection box," in Proc. European Conf. Computer Vision (ECCV), 2022. https://arxiv.org/abs/2110.06864

[3] L. Yang, B. Kang, Z. Huang et al., "Depth Anything V2," in Advances in Neural Information Processing Systems, 2024. https://arxiv.org/abs/2406.09414

[4] R. Ranftl, A. Bochkovskiy, and V. Koltun, "Vision transformers for dense prediction," in Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV), 2021. https://arxiv.org/abs/2103.13413

[5] Ultralytics, "Ultralytics YOLO documentation." [Online]. Available: https://docs.ultralytics.com/

[6] World Health Organization, World Report on Vision. Geneva: WHO, 2019. https://www.who.int/publications/i/item/9789241516570

[7] S. Khan, S. Nazir, and H. U. Khan, "Analysis of navigation assistants for blind and visually impaired people: A systematic review," IEEE Access, vol. 9, pp. 26712–26734, 2021. https://doi.org/10.1109/ACCESS.2021.3052415

[8] J. Xu et al., "Wearable obstacle avoidance electronic travel aids for blind and visually impaired individuals: A systematic review," IEEE Access, 2023. https://doi.org/10.1109/ACCESS.2023.3285396

[9] R. K. Katzschmann, B. Araki, and D. Rus, "Safe local navigation for visually impaired users with a time-of-flight and haptic feedback device," IEEE Trans. Neural Syst. Rehabil. Eng., vol. 26, no. 3, pp. 583–593, 2018. https://doi.org/10.1109/TNSRE.2018.2800665

[10] M. Eltaher et al., "A systematic review of urban navigation systems for visually impaired people," Sensors, vol. 21, no. 9, p. 3103, 2021. https://doi.org/10.3390/s21093103

[11] O. I. Johannesson et al., "The Sound of Vision project: The concept and first results," Brain Sciences, vol. 6, no. 3, p. 20, 2016. https://doi.org/10.3390/brainsci6030020

[12] P. B. Shull and D. D. Damian, "Haptic wearables as sensory replacement, sensory augmentation and trainer," J. NeuroEngineering and Rehabilitation, vol. 12, p. 59, 2015. https://doi.org/10.1186/s12984-015-0055-z

[13] V. Ranganeni, M. Sinclair, E. Ofek, A. Miller, J. Campbell, A. Kolobov, and E. Cutrell, "Exploring levels of control for a navigation assistant for blind travelers," in Proc. ACM/IEEE Int. Conf. Human-Robot Interaction, 2023, pp. 4–12. https://doi.org/10.1145/3568162.3578630

[14] C. Zhang et al., "An ARCore based user centric assistive navigation system for visually

impaired people," Applied Sciences, vol. 9, no. 5, p. 989, 2019.

[15] Mendeley Data, "Object detection dataset: Navigation assistance for the visually impaired people using YOLOv11," dataset ID m68g3h7p87. [Online]. Available: https://data.mendeley.com/

[16] OpenStreetMap contributors, "OpenStreetMap." [Online]. Available: https://www.openstreetmap.org/

[17] D. Luxen and C. Vetter, "Real-time routing with OpenStreetMap data," in Proc. 19th ACM SIGSPATIAL Int. Conf. Advances in Geographic Information Systems, 2011. See also https://project-osrm.org/

[18] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Advances in Neural Information Processing Systems, 2019.

[19] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000. https://opencv.org/

[20] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in Proc. EMNLP: System Demonstrations, 2020. https://huggingface.co/docs/transformers

[21] Expo, "Expo documentation." [Online]. Available: https://docs.expo.dev/

[22] Nominatim, "Nominatim documentation." [Online]. Available: https://nominatim.org/

[23] Leaflet, "Leaflet – an open-source JavaScript library for interactive maps." [Online]. Available: https://leafletjs.com/

[24] pytest, "pytest documentation." [Online]. Available: https://docs.pytest.org/